

MEMORIA DE PRUEBAS UNITARIAS

Módulo	Desarrollo de interfaces (2.º DAM)
Tarea	UD8 - Tarea 1
Alumno	Álvaro Jiménez Muñoz
Fecha	22/01/2026

Resumen de la práctica: En esta tarea se ha implementado un conjunto de pruebas unitarias automatizadas utilizando el framework MSTest de Visual Studio. El objetivo ha sido verificar la integridad funcional de la clase CalculadoraEjemplo, comprobando sus cuatro operaciones básicas (Suma, Resta, Multiplicación y División) mediante el patrón AAA (Arrange, Act, Assert), tanto en escenarios de éxito como de fallo controlado.

Indice de contenidos

1. Código fuente a probar.....	3
2. Clase de pruebas unitarias.....	4
3. Ejecución y verificación de resultados.....	5

1. Código fuente a probar

A continuación se muestra el código de la clase estática que contiene la lógica de negocio a verificar.

```
namespace Calculadora
{
    8 referencias | 0/2 passing
    public static class CalculadoraEjemplo
    {
        2 referencias | 0/2 passing
        public static int Suma (int item1, int item2)
        {
            return item1 + item2;
        }
        2 referencias | 0/2 passing
        public static int Resta(int item1, int item2)
        {
            return item1 - item2;
        }
        2 referencias | 0/2 passing
        public static int Multiplicacion(int item1, int item2)
        {
            return item1 * item2;
        }
        2 referencias | 0/2 passing
        public static double Division(int item1, int item2)
        {
            return item1 / item2;
        }
    }
}
```

Descripción: La clase CalculadoraEjemplo cuenta con cuatro métodos estáticos simples que reciben dos enteros y devuelven el resultado de la operación matemática correspondiente. En el caso de la división, se realiza un cast a double para permitir resultados decimales.

2. Clase de pruebas unitarias

Se ha creado un proyecto de pruebas separado (UnitTestProject...) que referencia a la librería principal.

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using Calculadora;

namespace UnitTestProject_AJM
{
    [TestClass]
    0 referencias
    public class UnitTest_AJM
    {
        [TestMethod]
        0 referencias
        public void TestSumaCorrecto()
        {
            // Arrange
            int a = 2;
            int b = 5;
            int esperado = 7;
            // Act
            int resultado = CalculadoraEjemplo.Suma(a, b);
            // Assert
            Assert.AreEqual(esperado, resultado);
        }

        [TestMethod]
        0 referencias
        public void TestSumaIncorrecto()
        {
            // Arrange
            int a = 2;
            int b = 5;
            int esperado = 9;
            // Act
            int resultado = CalculadoraEjemplo.Suma(a, b);
            // Assert
            Assert.AreEqual(esperado, resultado);
        }

        [TestMethod]
        0 referencias
        public void TestRestaCorrecto()
        {
            int resultado = CalculadoraEjemplo.Resta(10, 5);
            Assert.AreEqual(5, resultado);
        }
    }
}
```

```
[TestMethod]
0 referencias
public void TestRestaIncorrecto()
{
    int resultado = CalculadoraEjemplo.Resta(10, 5);
    Assert.AreEqual(0, resultado);
}

[TestMethod]
0 referencias
public void TestMultiplicacionCorrecto()
{
    int resultado = CalculadoraEjemplo.Multiplicacion(3, 3);
    Assert.AreEqual(9, resultado);
}

[TestMethod]
0 referencias
public void TestMultiplicacionIncorrecto()
{
    int resultado = CalculadoraEjemplo.Multiplicacion(3, 3);
    Assert.AreEqual(100, resultado);
}

[TestMethod]
0 referencias
public void TestDivisionCorrecto()
{
    double resultado = CalculadoraEjemplo.Division(10, 2);
    Assert.AreEqual(5, resultado);
}

[TestMethod]
0 referencias
public void TestDivisionIncorrecto()
{
    double resultado = CalculadoraEjemplo.Division(10, 2);
    Assert.AreEqual(1, resultado);
}
}
```

Explicación del Diseño: Se han diseñado 8 métodos de prueba (2 por cada operación):

- **Test Correcto:** Se introducen valores válidos y se compara el resultado con el valor real esperado (Ej: $2+5 = 7$). El `Assert.AreEqual` valida que la operación es correcta.
- **Test Incorrecto:** Se introducen valores y se comparan con un resultado erróneo intencionado (Ej: $2+5 = 9$) para verificar que el sistema de testing detecta la discrepancia y marca el error en rojo.

3. Ejecución y verificación de resultados

A continuación se muestra la ejecución de la batería completa de pruebas. Se pueden observar dos comportamientos distintos según el diseño del test.

Prueba	Duración	R.	Mensaje de error	Ejecutar Depurar
✖ UnitTestProject (8)	643 ms			Resumen del grupo UnitTestProject Pruebas en grupo: 8 ⌚ Duración total: 643 ms Salidas ✔ 4 Correcta ✖ 4 Con error
✖ UnitTestProject_AJM (8)	643 ms			
✖ UnitTest_AJM (8)	643 ms			
✔ TestDivisionCorrecto	37 ms			
✖ TestDivisionIncorrecto	112 ms		Error de Assert.AreEqual. Se esperaba <1>, pero es <5>.	
✔ TestMultiplicacionCorrecto	36 ms			
✖ TestMultiplicacionIncorrecto	155 ms		Error de Assert.AreEqual. Se esperaba <100>, pero es <9...	
✔ TestRestaCorrecto	36 ms			
✖ TestRestaIncorrecto	155 ms		Error de Assert.AreEqual. Se esperaba <0>, pero es <5>.	
✔ TestSumaCorrecto	< 1 ms			
✖ TestSumaIncorrecto	112 ms		Error de Assert.AreEqual. Se esperaba <9>, pero es <7>.	

Análisis de los resultados:

1. Casos de Éxito: Los métodos terminados en ...Correcto (como TestSumaCorrecto) han pasado la prueba satisfactoriamente.

- **Explicación:** Se introdujeron valores válidos (ej: 2+5) y se compararon con el resultado real esperado (7). Al coincidir, el Assert valida el código como correcto.

2. Casos de Fallo Controlado: Los métodos terminados en ...Incorrecto (como TestSumaIncorrecto) han fallado intencionadamente tal y como se diseñaron.

- **Explicación:** En estos tests forzamos un error esperando un resultado falso (ej: esperar un 9 cuando la suma es 7). El sistema de testing ha funcionado correctamente al detectar esta discrepancia y alertar del error.